Prompt de sistema con salida JSON estructurada (Bloque 10) / Prompt de sistema, no conversación: la diferencia real

✓ **Hecho:** Ver

Prompt de sistema, no conversación: la diferencia real

Con la **Unidad U1.4** cerrada, ya tienes una miniaplicación con arquitectura documentada: sabes cómo piensa el modelo, cuánto cuesta cada petición, y por qué elegiste el modelo que elegiste.

> **Prerrequisitos**: No necesitas instalar nada nuevo: vas a modificar el mismo módulo que ya tienes funcionando desde el Bloque 4, específicamente tu función "armar_peticion", con tu entorno virtual activo y tu archivo ".env" ya configurado.

Esta **unidad** da un giro distinto: en vez de mirar qué pasa alrededor de la petición (costo, latencia, arquitectura), vas a mirar qué va dentro de ella.

Empezamos por una pieza que, sin darte cuenta, vienes dejando fuera desde el **Bloque 4**: el **prompt** de sistema.

🔎 Lo que has estado haciendo hasta ahora

Cada vez que armaste una petición con "armar_peticion", pusiste todo dentro de "contents": tanto la pregunta real del usuario como, probablemente, alguna instrucción de comportamiento que se te ocurrió mezclar ahí mismo, como "responde de forma breve" o "actúa como un asistente técnico".

Funciona, porque el modelo igual lee ese texto y lo obedece. Pero mezclar instrucción y contenido en el mismo lugar tiene un costo que todavía no habías tenido que pagar.

⚠ Ese costo aparece apenas tu aplicación necesita hacer la misma tarea muchas veces, con datos distintos cada vez: si la instrucción de comportamiento vive pegada al dato de entrada, tienes que reescribirla (o al menos repetirla) en cada petición, y cualquier variación accidental en cómo la escribiste esta vez, comparado con la anterior, puede cambiar el comportamiento del modelo sin que lo hayas decidido tú.

❓ Qué es exactamente un prompt de sistema

Un **prompt de sistema es una instrucción** que se envía en un canal separado del mensaje del usuario, pensada para fijar el comportamiento del modelo de forma estable: quién es, cómo debe responder, qué formato tiene que usar, qué cosas no debe hacer.

No es parte de la conversación visible; es una capa de configuración que se aplica antes de que el modelo procese lo que el usuario realmente preguntó.

En la **API de Gemini**, esta separación existe como un campo propio de la petición, **"systemInstruction"**, distinto de **"contents"**.

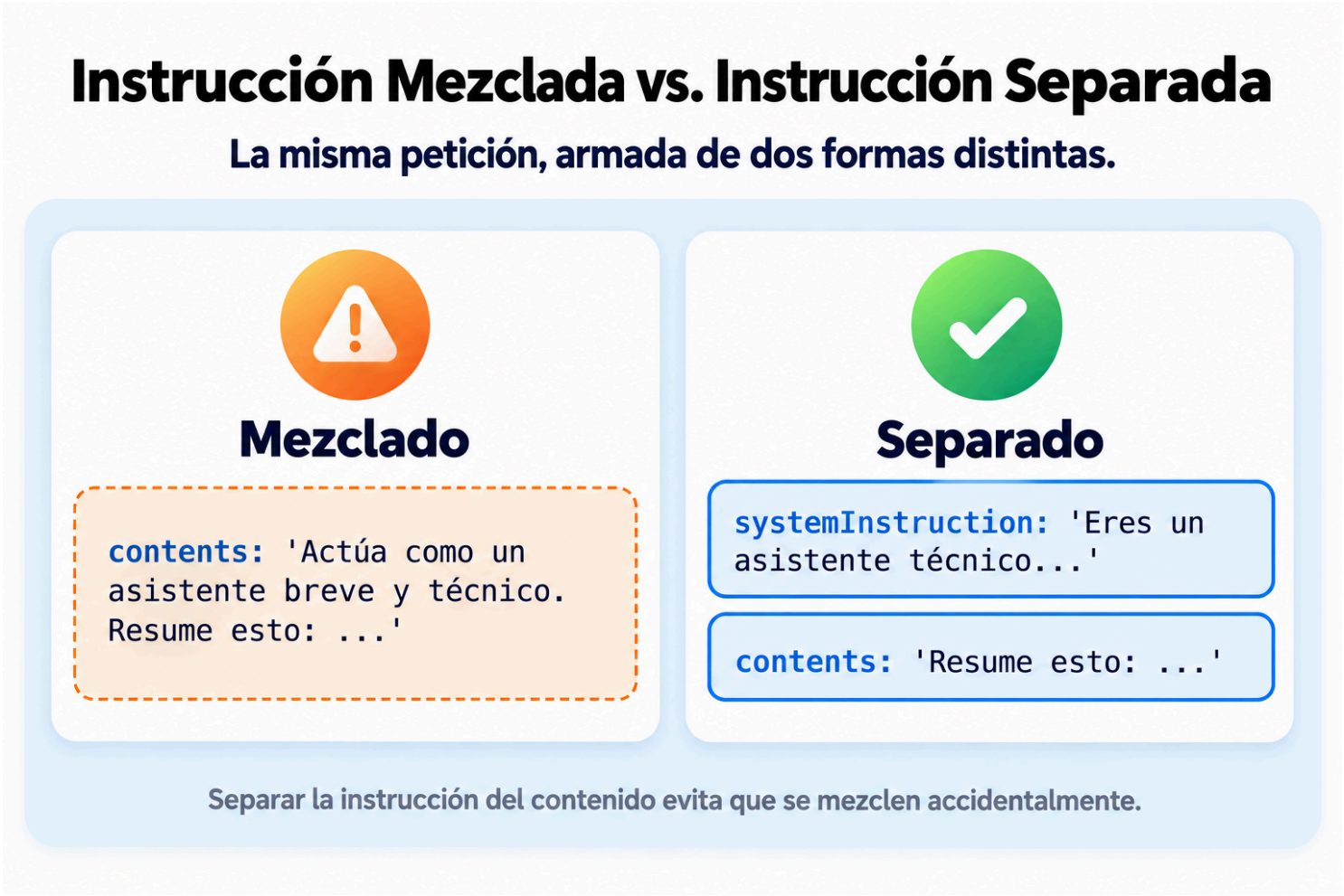
Así se ve la diferencia entre lo que probablemente venías haciendo y lo que vas a hacer desde ahora:

```
# Antes: instrucción y pregunta mezcladas en el mismo texto
 {
   "contents": [
     { "parts": [ { "text": "Actúa como un asistente breve y técnico. Resume esto: ..." } ]
   ]
 }

 # Ahora: instrucción separada de la pregunta
 {
   "systemInstruction": {
     "parts": [ { "text": "Eres un asistente técnico. Responde siempre en un máximo de 3 or
   },
   "contents": [
     { "parts": [ { "text": "Resume esto: ..." } ] }
   ]
 }
```

El nombre exacto del campo **("systemInstruction")** es específico de **Gemini**. Otros **proveedores** lo resuelven distinto (por ejemplo, un mensaje con rol **"system"** en la lista de mensajes), pero el concepto de

fondo, un canal separado para instrucciones estables, es prácticamente universal en las **APIs** de modelos de lenguaje que vas a encontrar en el resto del programa.



🧩 Por qué separarlo no es solo prolijidad

Se repite sin copiar y pegar

El **prompt** de sistema se arma una sola vez en tu código y se reutiliza en cada petición, sin que tengas que reescribirlo (ni arriesgarte a escribirlo distinto) cada vez que cambia la pregunta del usuario.

Es más difícil de ignorar

El modelo trata las instrucciones de sistema con más peso que el texto normal del usuario, precisamente porque están en un canal distinto, pensado para eso. Una instrucción de formato mezclada dentro de un mensaje largo compite por atención con el resto del texto; una instrucción de sistema no.

> **Separa lo que cambia de lo que no**
>
> La instrucción de comportamiento ("**responde en JSON**", "**actúa como...**") es casi siempre igual en todas las llamadas de una misma funcionalidad; la pregunta del usuario cambia en cada llamada. Mantenerlas separadas hace que tu código refleje esa diferencia en vez de esconderla dentro de un solo bloque de texto.

🛠️ Cómo se traduce esto en tu módulo

Llevar esto a tu código es un cambio pequeño en **"armar_peticion"**: en vez de recibir un solo texto y meterlo todo en **"contents"**, la función ahora recibe también un **"system_instruction"** separado, y arma el cuerpo de la petición con los dos campos distintos.

```
def armar_peticion(pregunta_usuario, system_instruction):
    api_key = os.getenv("GEMINI_API_KEY")
    modelo = "gemini-3.6-flash"
    url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConter
    body = {
        "systemInstruction": {
            "parts": [ { "text": system_instruction } ]
        },
        "contents": [
            { "parts": [ { "text": pregunta_usuario } ] }
        ]
    }
    return url, body
```

El resto de tu módulo no cambia: **"enviar_peticion"** sigue mandando el mismo cuerpo JSON que ya construías, solo que ahora ese cuerpo trae una pieza más, y **"extraer_texto"** sigue leyendo la respuesta exactamente igual que antes.

Una aclaración importante: esto sigue siendo la API REST directa, la misma que ya vienes usando con "requests" desde el Bloque 4, construyendo la URL y el cuerpo JSON a mano. No es el SDK oficial de Python de Google ("google-genai"). El concepto de instrucción de sistema existe igual en ese SDK, pero se pasa de una forma distinta a como armas este diccionario aquí. No es necesario que lo resuelvas ahora: solo que sepas que "armar_peticion", tal como está construida en este módulo, es específica del camino REST que ya venías siguiendo.

Buenas prácticas

En proyectos más grandes es común agregar **"type hints"** a funciones como esta, por ejemplo **"def armar_peticion(pregunta_usuario: str, system_instruction: str) -> tuple[str, dict]:"**. Ninguna función de tu módulo los usa todavía, así que no hace falta que empieces por aquí: es una práctica para tener presente más adelante, no un requisito de esta lección.

Así se ve el flujo completo, de punta a punta, usando tus tres funciones ya conocidas:

```
pregunta = "Explica qué es una API en términos simples."
system_instruction = "Eres un asistente técnico. Responde siempre en un máximo de 3 oracior

url, body = armar_peticion(pregunta, system_instruction)
respuesta_json = enviar_peticion(url, body)
texto = extraer_texto(respuesta_json)
print(texto)
```

Con una petición real, algo así (el texto exacto varía en cada llamada, pero siempre respeta el límite de 3 oraciones del prompt de sistema):

```
Una API es una interfaz que permite que dos programas se comuniquen entre
sí, generalmente enviando y recibiendo datos según reglas definidas. Actúa
como un contrato que especifica qué se puede pedir y qué se puede esperar
a cambio. Es la forma estándar en la que las aplicaciones modernas se
conectan con servicios externos.
```

Posibles errores comunes

Si escribes mal el nombre del campo dentro del diccionario, por ejemplo **"system_instruction"** (con guion bajo, el nombre de tu parámetro de Python) en vez de **"systemInstruction"** (con mayúscula intermedia, el nombre real del campo JSON), la API no lo ignora en silencio: responde con un error HTTP **400** y un mensaje del tipo **"Invalid JSON payload received. Unknown name 'system_instruction': Cannot find field."**, nombrando exactamente el campo que no reconoce.

🔎 **Importante:** es un 400 distinto al que viste en la unidad anterior por exceder la ventana de contexto: ahí el problema era el tamaño de la petición, aquí es que el cuerpo JSON no tiene la

forma que la API espera. Si ves este mensaje, revisa la ortografía exacta de las claves de tu diccionario contra la de esta lección.

🧭 Qué va en el prompt de sistema, y qué no

No toda instrucción pertenece al prompt de sistema.

> 📌 **La regla práctica es simple**: si la instrucción es la misma sin importar qué pregunte el usuario, va en el sistema; si depende de la pregunta puntual, va en "contents".



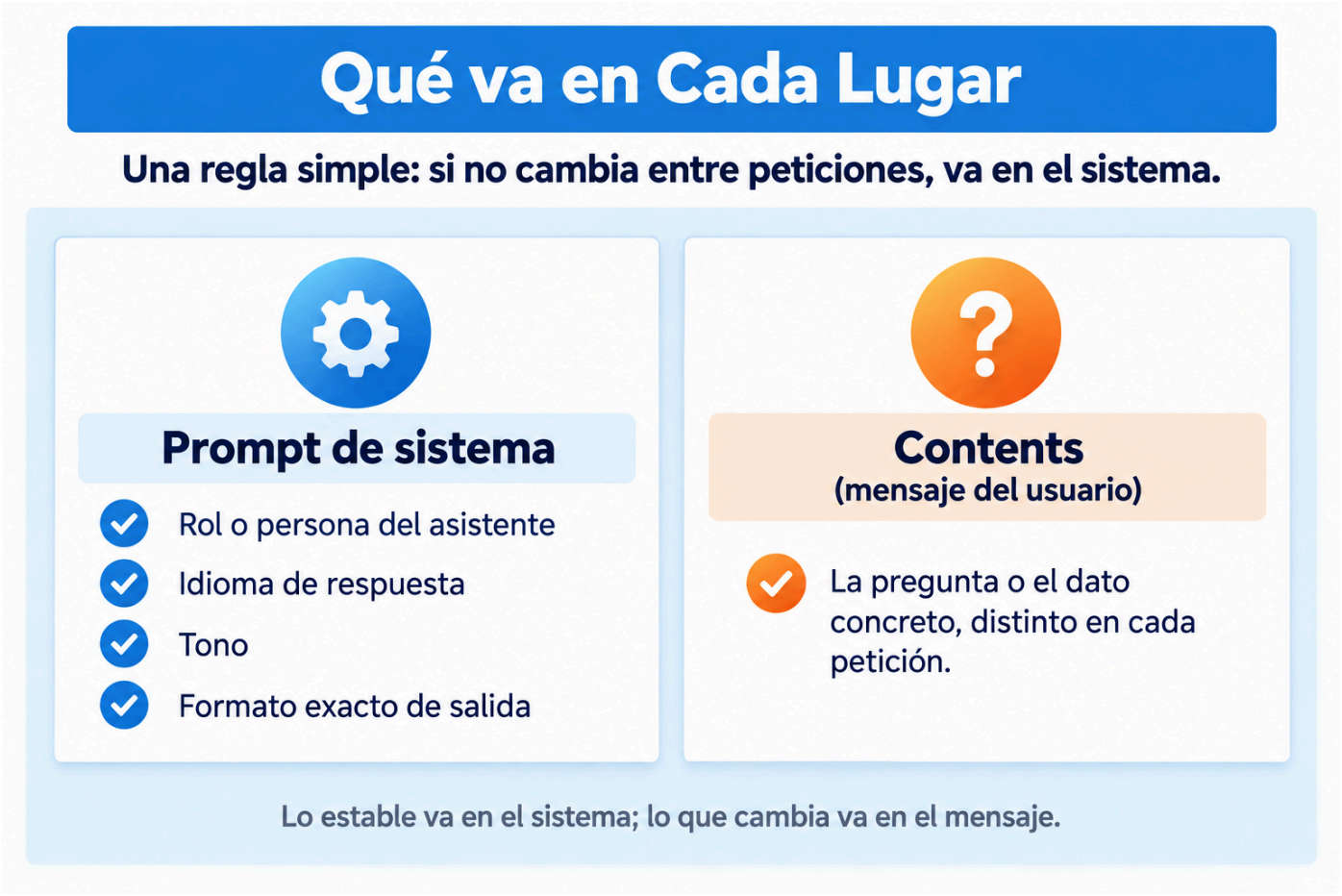
Va en el prompt de sistema

El rol o persona del asistente, el idioma de respuesta, el tono, las restricciones de longitud, y (lo que vas a usar en la siguiente lección) el formato exacto de salida.



Va en "contents"

La pregunta o el dato concreto que el usuario quiere procesar en esa petición puntual, distinto cada vez.



Siguiente lección

Bloque 10 · Formato de salida JSON

Con esta separación ya clara en tu código, en la siguiente lección vas a usar exactamente este mecanismo para resolver un problema muy concreto: pedirle al modelo que su respuesta venga en un formato **JSON** estructurado, no en texto libre, y qué hacer en los casos en que el modelo no coopera.

Última modificación: jueves, 24 de septiembre de 2026, 16:48